

Roll Number

--	--	--	--	--	--	--

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking



Partially Filled



Lightly filled



Analysis & Design of Algorithms 2026 — Endsem

Duration: 2 hour

Total Marks: $6 \times 10 = 60$

Name: _____

Roll No.: _____

Problem 1. Given a *directed acyclic graph*, design a linear time algorithm to determine, for every vertex v , the *number of distinct paths* from a given vertex s to v . Your algorithm just needs to return the numbers for each vertex and not the actual paths. As always, no matter what algorithm you can think of, it is a DP in disguise. So, we need subproblem definitions (1 mark), recurrences (2 marks), pseudocode (4 marks) and runtime analysis (2 marks). You may need a pre-processing before defining or running the DP - state that clearly (1 points)

Solution and Rubric for Q1

Pre-processing (1 Mark)

Run a topological sort on the directed acyclic graph (DAG) G to produce an ordering $v_1, v_2, \dots, v_n$ such that every edge (u, v) satisfies u appearing before v . This takes $\mathcal{O}(n + m)$ time using a DFS-based topological sort. Processing the vertices in this order ensures that all predecessors of a vertex are processed before the vertex itself.

Rubric [1.0 Mark]: Awarded for correctly identifying the need for a Topological Sort to establish a valid processing order.

Subproblem Definition (1 Mark)

For each vertex $v \in V$, let $P(v)$ denote the number of distinct directed paths from the source s to v in G .

Rubric [1.0 Mark]: Awarded for clearly defining the state variable (e.g., $P(v)$ or $DP[v]$) as the number of distinct directed paths from s to v .

Recurrence (2 Marks)

Base Case: $P(s) = 1$ (the empty path from s to itself).

Recursive Step: For every other vertex $v \neq s$, every path from s to v ends with some edge $(u, v) \in E$, and the prefix is a path from s to u . Different incoming edges (or different paths to u) give different paths to v . Therefore:

$$P(v) = \sum_{(u,v) \in E} P(u)$$

with $P(v) = 0$ if v is unreachable from s .

Because G is a DAG, processing vertices in topological order guarantees that all $P(u)$ on the right-hand side are already computed when we evaluate $P(v)$.

Rubric Breakdown:

- **[0.5 Marks]:** Base Case (explicitly stating $P(s) = 1$).

- **[1.0 Mark]:** Recursive Step (correctly defining the mathematical relationship $P(v) = \sum P(u)$).
- **[0.5 Marks]:** Handling Unreachable Nodes (accounting for nodes that cannot be reached from s , e.g., stating $P(v) = 0$).

Pseudocode (4 Marks)

Algorithm 1 CountPaths(G, s)

```

1: Compute a topological order  $v_1, v_2, \dots, v_n$  of  $G$   $\triangleright \mathcal{O}(n + m)$ 
2: Initialize  $P[v] = 0$  for all  $v \in V$ 
3:  $P[s] = 1$ 
4: for  $i = 1$  to  $n$  do  $\triangleright$  Process in topological order
5:    $u = v_i$ 
6:   for each edge  $(u, w)$  in outgoing edges of  $u$  do
7:      $P[w] = P[w] + P[u]$   $\triangleright$  Pushing the paths forward
8:   end for
9: end for
10: return  $P$ 

```

Note: Both "pushing" to outgoing edges (shown above) or "pulling" from incoming edges are correct $\mathcal{O}(n + m)$ DP implementations.

Rubric Breakdown:

- **[1.0 Mark]:** Initialization (correctly initializing the array: 0 for all v , 1 for s).
- **[1.0 Mark]:** Outer Loop (iterating through vertices strictly following the computed topological order).
- **[1.5 Marks]:** State Transition (accurately implementing the recurrence relation; accepts both "pull" and "push" methods).
- **[0.5 Marks]:** Return (returning the final array/structure).

Runtime Analysis (2 Marks)

Topological sort takes $\mathcal{O}(n + m)$ time. Initialization takes $\mathcal{O}(n)$ time. The main loop visits each vertex exactly once and, for each vertex, iterates over its outgoing edges. In total, this examines every edge exactly once, so the loop costs $\mathcal{O}(n + m)$.

Total Runtime: $\mathcal{O}(n + m)$, which is linear as required.

Rubric Breakdown:

- **[1.0 Mark]:** Topological Sort (noting that computing the topological order takes $\mathcal{O}(n + m)$).
- **[1.0 Mark]:** DP Traversal (explaining that processing vertices and edges takes $\mathcal{O}(n + m)$, concluding with a correct total linear runtime).

Problem 2. You are given a directed graph $G = (V, E)$ with positive edge weights, a source vertex s , a destination vertex t , and a mandatory intermediate vertex v . The goal is to find the shortest *simple* path from s to t that passes through v (recall that a simple path visits each vertex at most once).

- (a) Suppose G is a directed acyclic graph (DAG). Give an $O(m \log n)$ time algorithm to find the shortest simple path from s to t that passes through v . Just describe the algorithm in English briefly but precisely and argue runtime [5 marks]

Solution and Rubric for Q2a

(a) Algorithm for DAGs in $O(m \log n)$ (5 marks)

Idea : Not needed for marking In a DAG, any path from s to t that passes through v splits naturally into an $s \rightarrow v$ sub-path followed by a $v \rightarrow t$ sub-path. Crucially, because the graph is acyclic, the two sub-paths cannot share any vertex other than v (a shared vertex $w \neq v$ would create a directed cycle $w \rightarrow \dots \rightarrow v \rightarrow \dots \rightarrow w$). Therefore the concatenation is automatically a simple path, and the optimal answer is exactly:

$$d_1(v) + d_2(v),$$

where $d_1(v) = \text{dist}(s, v)$ and $d_2(v) = \text{dist}(v, t)$ in the original DAG.

Algorithm.

- 1 Run Dijkstra's algorithm from s in G , obtaining $d_1(v)$, the shortest-path distance from s to v .
- 2 Build the reverse graph G^R (reverse every edge, keep weights). Run Dijkstra from t in G^R , obtaining $d_2(v)$, the shortest-path distance from v to t in G . (*Note: Running Dijkstra directly from v to t in G is an equally valid alternative*).
- 3 If either $d_1(v) = \infty$ or $d_2(v) = \infty$, output "no such path exists." Otherwise output $d_1(v) + d_2(v)$, and the path obtained by concatenating the two Dijkstra trees at v .

Runtime. Two Dijkstra runs on a graph with n vertices and m edges with positive weights, using a binary heap, cost $O((n+m) \log n) = O(m \log n)$ each (assuming $m \geq n-1$). Reversing the graph is $O(n+m)$.

Total: $O(m \log n)$.

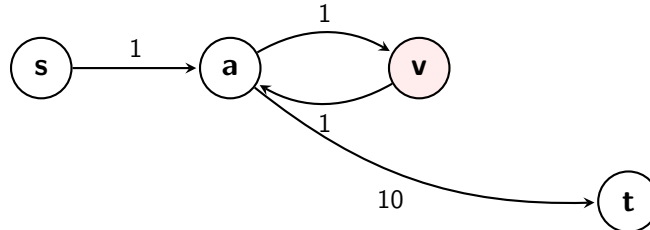
Rubric Breakdown:

- **[1.0 Marks] Algorithm Steps:** Proposing two shortest-path searches (e.g., Dijkstra from s to v , and Dijkstra from v to t).
 - **[2.0 Mark] Output Generation:** [1.0 mark] for Correctly combining the distances and handling the unreachable edge cases (checking for ∞). [1.0 mark] for writing (even at a high level) how to take the union of the paths.
 - **[2.0 Mark] Runtime Analysis:** Justifying the $O(m \log n)$ runtime based on the chosen pathfinding algorithm. 2.0 mark for clearly stating the runtime of Dijkstra.
- (b) Give a concrete example of a directed graph with positive edge weights, with specific vertices s, t, v , where the above algorithm will produce a wrong output. [5marks]

Solution and Rubric for Q2b

On cyclic graphs, the two shortest paths can share vertices other than v , so their concatenation may not be simple.

Counter-example Graph: (5 marks)



What the algorithm computes:

- $d_1(v)$ = shortest $s \rightarrow v = s \rightarrow a \rightarrow v = 2$.
- $d_2(v)$ = shortest $v \rightarrow t = v \rightarrow a \rightarrow t = 11$.

It reports $2 + 11 = 13$, with witness path $s \rightarrow a \rightarrow v \rightarrow a \rightarrow t$.

Why this is wrong: The concatenated witness path visits vertex a twice, violating the simple path constraint. The actual shortest *simple* path does not exist in this graph.

Rubric Breakdown:

- [2.0 Marks] Graph Construction: Clear example with a cycle and specified s, t, v .
- [1.0 Mark] Algorithm Trace: Correctly calculating $d_1(v)$ and $d_2(v)$.
- [2.0 Marks] Failure Explanation: Demonstrating the concatenated path is not simple.

Problem 3. Consider the following problem. You are given a flow network with unit-capacity edges: it consists of a directed graph $G = (V, E)$, a source s and a sink t , and $u_e = 1$ for every edge e . You are also given a parameter k . The goal is to delete k edges so as to reduce the maximum s - t flow in G by as much as possible. In other words, you should find a set of edges $F \subseteq E$ so that $|F| = k$ and the maximum s - t flow in the graph $G' = (V, E \setminus F)$ is as small as possible.

- (a) Give a polynomial-time algorithm to solve this problem. A precise English description is sufficient. Proof of correctness not required. [5 marks]

Solution and Rubric for Q3a

Key observation : Not needed for marking In a unit-capacity network, by Menger's theorem, the max $s - t$ flow equals the minimum number of edges whose removal disconnects s from t (i.e., the size of a minimum $s - t$ edge cut). Let $F^* = \text{max-flow}(G) = \text{min-cut}(G)$.

If we delete any set F of k edges, the new max-flow drops by at most k (each deleted edge had capacity 1, so the total decrease is bounded by $|F| = k$). It can drop by exactly k if and only

if some min-cut of size $F^* - k$ exists in the new graph; equivalently, we want F to be a subset of k edges of G that lies strictly inside some $s - t$ min-cut.

Algorithm.

- (a) Compute a minimum $s - t$ edge cut C in G (e.g., by running max-flow with Dinic's or Edmonds-Karp algorithm, then taking the set of edges crossing from the reachable s -component to the rest of the graph). Let $F^* = |C|$.
- (b) If $k \geq F^*$: delete all F^* edges of C (and any other $k - F^*$ edges arbitrarily). The new max-flow is 0, which is the smallest possible.
- (c) If $k < F^*$: output any k edges of the min-cut C . Then the new graph G' has a max-flow of exactly $F^* - k$, which is the best achievable reduction.

Runtime. One max-flow computation on unit-capacity graphs using Ford-Fulkerson can take $\mathcal{O}(mn)$ time. Recovering the min-cut and selecting k edges takes $\mathcal{O}(m)$ time.

Rubric Breakdown:

- **[1.0 Mark]** Cut Computation: Specifying the use of a standard algorithm to find a min-cut.
- **[2.0 Marks]** Deletion Strategy: Correctly handling both cases ($k < |C|$ and $k \geq |C|$) by arbitrarily deleting k edges from the found min-cut. [1.0 mark] for each case.
- **[1.0 Mark]** Runtime Analysis: Correctly identifying the polynomial runtime of the chosen flow algorithm.

Algorithm Trace for Unit Capacities (Demonstration):

Suppose we have a diamond graph (s, a, b, t) where $(s, a), (s, b), (a, t), (b, t)$ all have capacity 1. Max flow is 2. The min-cut C is $\{s \rightarrow a, s \rightarrow b\}$. If $k = 1$, the algorithm arbitrarily picks any edge in C , say (s, a) . Deleting it drops the max flow to exactly 1 (via $s \rightarrow b \rightarrow t$). Because all capacities are 1, arbitrary selection from the min-cut is proven optimal.

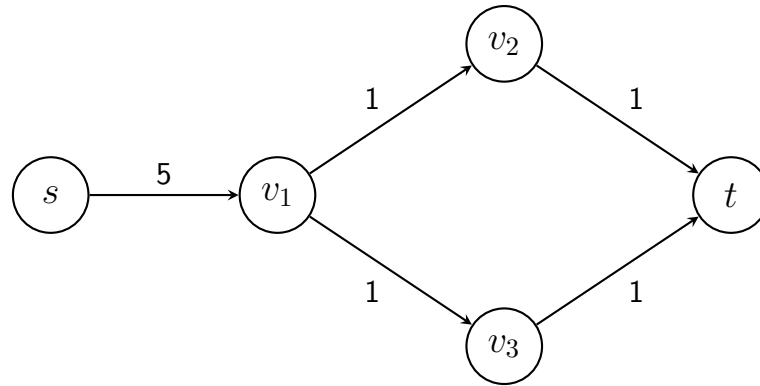
- (b) Give an example where your approach above fails for graphs with non-unit integer capacities with proper explanation. [5 marks]

Solution and Rubric for Q3b

With unit capacities, every edge in a min-cut contributes exactly 1, so a greedy choice of “delete *any* k edges from a min-cut” guarantees an optimal drop. With non-unit capacities, this assumption breaks down. The optimal edge to delete to maximize the flow reduction might not belong to *any* minimum cut at all.

To prove this, we construct a graph with a “single point of failure” that exists outside the minimum $s - t$ cut.

Counter-example Graph: Let $k = 1$.



What the algorithm computes:

- **Total Max Flow:** The maximum flow is **2** (1 unit through v_2 and 1 unit through v_3).
- **Minimum Cut:** The algorithm computes a minimum $s - t$ cut. A valid min-cut is $C = \{v_1 \rightarrow v_2, v_1 \rightarrow v_3\}$, which has a total capacity of $1 + 1 = 2$. (Note: The cut $\{s \rightarrow v_1\}$ has a capacity of 5, so it is mathematically ignored by the min-cut algorithm).
- **Algorithm's Action:** The algorithm states to delete *any* $k = 1$ edge from C . Suppose it arbitrarily picks the edge $v_1 \rightarrow v_2$.
- **Algorithm's Result:** After deleting $v_1 \rightarrow v_2$, the new max-flow is **1**. The flow dropped by exactly **1**.

Why this is sub-optimal: The optimal strategy to reduce the flow as much as possible with $k = 1$ is to ignore the min-cut entirely and delete the edge $s \rightarrow v_1$ (capacity 5). If we delete $s \rightarrow v_1$, the source is completely disconnected, and the new max flow becomes **0**.

Thus, the optimal deletion achieves a flow drop of **2**. Because the algorithm restricts its choices to the min-cut, it only achieved a flow drop of 1. This definitively proves the algorithm is sub-optimal for non-unit capacities.

Rubric Breakdown [5.0 Marks]:

- **[1.0 Mark]** Core Insight: Explaining that the optimal edge to delete in a non-unit network may not belong to a min-cut.
- **[2.0 Marks]** Graph Construction: Providing a clear graph with a bottleneck outside the min-cut (e.g., $s \rightarrow v_1$ with capacity 5 vs a min-cut of capacity 2).
- **[1.0 Mark]** Algorithm Trace: Showing the algorithm incorrectly selects an edge from the min-cut, resulting in a minimal flow drop (a drop of 1).
- **[1.0 Mark]** Optimal Trace: Identifying the high-capacity single point of failure ($s \rightarrow v_1$) that achieves the maximum flow reduction (a drop of 2), stating the algorithm's failure.

Problem 4. A city has n students who each need to be enrolled in one of k schools. The admissions are constrained in the following ways: (i) a student can only be enrolled in a school that is within distance d of their home, (ii) each school j has a capacity C_j , and no more than C_j students can be enrolled in school j . Design a polynomial (in terms of k, n) time algorithm that decides whether it is possible for every student to get an admission. No pseudocode needed but any constructions have to be *written* clearly. Only vague diagrams will fetch ZERO. Argue the precise runtime of your algorithm clearly.

Solution and Rubric for Q4

We model this as a max-flow computation on the following network N .

Construction

1. **Vertices.** Create a source s and a sink t . For each student $i \in \{1, \dots, n\}$ create a node u_i . For each school $j \in \{1, \dots, k\}$ create a node w_j .
2. **Source edges.** For every student i , add an edge $s \rightarrow u_i$ with capacity 1 (each student needs to be enrolled in exactly one school).
3. **Eligibility edges.** For every student-school pair (i, j) such that the distance from student i 's home to school j is at most d , add an edge $u_i \rightarrow w_j$ with capacity 1. (Pairs with distance $> d$ get no edge — this enforces constraint (i).)
4. **Sink edges.** For every school j , add an edge $w_j \rightarrow t$ with capacity C_j . (This enforces constraint (ii): no school can absorb more than its capacity.)

Rubric Breakdown [7.0 Marks]:

- **[2.0 Mark] Vertices & Source Edges:** Correctly setting up the bipartite nodes and connecting the source to students with capacity 1.
- **[2.0 Marks] Eligibility Edges:** Correctly linking students to schools conditionally based on the distance d , with capacity 1.
- **[3.0 Marks] Sink Edges:** Correctly connecting schools to the sink with capacities matching the school limit C_j .

Decision Rule

Compute the maximum $s-t$ flow F in N . Output **YES** (every student can be admitted) iff $F = n$; otherwise output **NO**.

Rubric Breakdown [1.0 Mark]:

- **[1.0 Mark]** Clearly stating that the max flow must exactly equal n (the total number of students) for a valid total assignment.

Size of the network and runtime

The network has $|V| = n + k + 2$ vertices and at most $|E| \leq n + nk + k = \mathcal{O}(nk)$ edges (the eligibility edges dominate).

Total flow value is bounded by n . Using the Ford-Fulkerson method with augmenting paths, each augmenting path increases the flow by at least 1 (integer capacities), so we run at most n BFS/DFS iterations, each costing $\mathcal{O}(|V| + |E|) = \mathcal{O}(nk)$.

Total runtime: $\mathcal{O}(n \cdot nk) = \mathcal{O}(n^2k)$, which is polynomial in n and k .

Rubric Breakdown [2.0 Marks]:

- **[1.0 Mark] Graph Size:** Accurately stating the bounds for $|V|$ and $|E|$ in terms of n and k .
- **[1.0 Mark] Polynomial Runtime:** Deriving the runtime using a standard flow algorithm (e.g., Ford-Fulkerson or Dinic's) and explicitly stating that the final bound is polynomial.

Problem 5. Define The Zero Weight Cycle problem as follows: you are given a directed graph $G = (V, E)$, with weights w_e on the edges $e \in E$. The weights can be negative or positive integers (or equal to 0). You need to decide if there is a cycle in G so that the sum of the edge weights on this cycle is exactly 0 (i.e., say YES if there is such a cycle, NO otherwise). Prove that the Subset Sum problem is polynomial time reducible to the Zero Weight Cycle Problem.

- (a) Describe the reduction from a given instance of Subset-Sum to an instance of Zero Weight Cycle [4 marks] Given an instance $(a_1, \dots, a_n; T)$ of Subset-Sum, build a directed graph G as follows:

- **Vertices.** Create $n + 1$ vertices : $v_0, v_1, v_2, \dots, v_n$. [1 mark]
 - **“Forward” edges (one layer per item).** For each item $i \in \{1, \dots, n\}$, add **two** parallel directed edges from v_{i-1} to v_i :
 - a “take” edge of weight a_i , and
 - a “skip” edge of weight 0.
- [2 marks] **Remark.** Many of you, instead of taking parallel edges, may have introduced a ‘dummy’ vertex for the skip edges. That is absolutely fine since its only a polynomial blow-up.
- **“Closing” edge.** Add a single directed edge from v_n back to v_0 with weight $-T$. [1 mark]

The construction has $n + 1$ vertices and $2n + 1$ edges, computable in polynomial time.

- (b) Prove that Yes instance of Subset-Sum implies an Yes instance of Zero Weight Cycle. [3 marks]

Suppose $S \subseteq \{1, \dots, n\}$ satisfies $\sum_{i \in S} a_i = T$. Construct a directed cycle in G as follows: starting at v_0 , for each $i = 1, 2, \dots, n$ traverse the $v_{i-1} \rightarrow v_i$ edge of weight a_i if $i \in S$, otherwise traverse the parallel edge of weight 0. After step n we are at v_n ; close the cycle with the back-edge $v_n \rightarrow v_0$ of weight $-T$. [2 marks for this. 1 mark for for stating each of the cases]

The total weight of this cycle is

$$\sum_{i \in S} a_i + \sum_{i \notin S} 0 + (-T) = T - T = 0.$$

Hence G has a zero-weight cycle. [1 mark]

- (c) Prove that Yes instance of Zero Weight Cycle implies an Yes instance of Subset-Sum. [3 marks]

Suppose there is a zero weight cycle in G . Since (v_{n+1}, v_1) is the only edge with negative length $-T$, this cycle must contain this edge. [1 mark] Therefore there is a path from v_1 to v_{n+1} in this cycle for which the weights of edges add up to exactly T . [1 mark]

Now, consider those v_i for which this path uses the ‘take’ edge (v_i, v_{i+1}) . The corresponding numbers x_i add up to T . [1 mark]

Problem 6. Suppose you are given a set of n currencies and an $n \times n$ matrix R of exchange rates, where $R[i, j]$ is the amount of currency j you can buy with exactly one unit of currency i . An arbitrage opportunity is a cycle of currency exchanges that yields a profit. Mathematically, it is a sequence of currencies $i_1, i_2, \dots, i_k$ where $i_1 = i_k$, such that the product of their exchange rates is strictly greater than 1:

$$R[i_1, i_2] \times R[i_2, i_3] \times \dots \times R[i_{k-1}, i_k] > 1$$

Design an efficient algorithm to determine if an arbitrage opportunity exists in the given market. (Hint: Reduce it to a known problem.)

Solution and Rubric

Algorithm [8 marks]

1. Build a directed graph $G = (V, E)$
 - one vertex per currency, $V = \{1, 2, \dots, n\}$. [1 mark]
 - For every ordered pair (i, j) with $i \neq j$ and $R[i, j] > 0$, add a directed edge $i \rightarrow j$ [1 mark]
 - with weight

$$w(i, j) = -\log R[i, j].$$

[2 mark] The base of the logarithm does not matter (it just rescales all weights by the same positive constant, preserving signs and zeroes).

2. Add a virtual source vertex s with a 0-weight edge $s \rightarrow i$ for every i . (This guarantees every vertex is reachable, so Bellman–Ford can detect any negative cycle in the graph, regardless of whether it is reachable from a fixed currency.) [2 marks]
3. Run **Bellman–Ford** from s . After n rounds of edge relaxation, perform one extra round: if any edge (u, v) can still be relaxed (i.e. $d[u] + w(u, v) < d[v]$), report **arbitrage exists**. Otherwise report **no arbitrage**. [2 marks]

Runtime [2 marks]

$|V| = n + 1$ and $|E| \leq n^2 + n = O(n^2)$. Bellman–Ford runs in

$$O(|V| \cdot |E|) = O(n \cdot n^2) = \boxed{O(n^3)},$$

which is polynomial. Constructing G from the matrix R takes $O(n^2)$ additional time. (1 mark for each of the above points)